HOME WORK-4

1.Implement a stack using queue data structure

```
#include <stdio.h>

#define MAX 100

int queue[MAX];
int front = 0, rear = -1;

void push(int x)
{
    int i, temp;

    if (rear == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }


    queue[++rear] = x;


    for (i = front; i < rear; i++)
    {
        temp = queue[i];
        queue[i] = queue[rear];
        queue[rear] = temp;
    }
}

void pop()
{
    if (front > rear)
    {
        printf("Stack Underflow\n");
        return;
    }

    printf("Popped element = %d\n", queue[front++]);
}

void display()
{
    int i;

    if (front > rear)
    {
        printf("Stack is empty\n");
        return;
```

```
    }

    printf("Stack: ");

    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);

    printf("\n");
}

int main()
{
    int choice, x;

    while (1)
    {
        printf("\n1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice == 1)
        {
            printf("Enter element: ");
            scanf("%d", &x);
            push(x);
        }
        else if (choice == 2)
        {
            pop();
        }
        else if (choice == 3)
        {
            display();
        }
        else if (choice == 4)
        {
            break;
        }
        else
        {
            printf("Invalid choice\n");
        }
    }

    return 0;
}
```

```
OUTPUT:
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 5

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 6

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack: 6 5

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Popped element = 6

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 4

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack: 4 5
--------------------------------------------------------------------------------
--------
2.Implement a stack using queue
#include <stdio.h>
#define MAX 5

int s1[MAX], s2[MAX];
int top1 = -1, top2 = -1;
```

```
void enqueue(int x)
{
    if (top1 == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    s1[++top1] = x;
}

void dequeue()
{
    int x;

    if (top1 == -1 && top2 == -1)
    {
        printf("Queue Underflow\n");
        return;
    }

    if (top2 == -1)
    {
        while (top1 != -1)
        {
            s2[++top2] = s1[top1--];
        }
    }

    x = s2[top2--];
    printf("Deleted element = %d\n", x);
}

void display()
{
    int i;

    printf("Queue: ");

    for (i = top2; i >= 0; i--)
        printf("%d ", s2[i]);

    for (i = 0; i <= top1; i++)
        printf("%d ", s1[i]);

    printf("\n");
}

int main()
```

```
{
    int choice, x;

    while (1)
    {
        printf("\n1.Enqueue  2.Dequeue  3.Display  4.Exit\n");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter element: ");
                scanf("%d", &x);
                enqueue(x);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;
        }
    }
}
```

OUTPUT:

```
1.Enqueue  2.Dequeue  3.Display  4.Exit
1
Enter element: 3

1.Enqueue  2.Dequeue  3.Display  4.Exit
1
Enter element: 7

1.Enqueue  2.Dequeue  3.Display  4.Exit
2
Deleted element = 3

1.Enqueue  2.Dequeue  3.Display  4.Exit
3
Queue: 7

1.Enqueue  2.Dequeue  3.Display  4.Exit
4
```

3.Browser history using stack

```
#include <stdio.h>
#include <string.h>

#define MAX 10

char history[MAX][50];
int top = -1;

void visit(char page[])
{
    if (top == MAX - 1)
        printf("History is full\n");
    else
    {
        strcpy(history[++top], page);
        printf("Visited: %s\n", page);
    }
}

void back()
{
    if (top <= 0)
        printf("No previous page\n");
    else
    {
        top--;
        printf("Back to: %s\n", history[top]);
    }
}

void current()
{
    if (top == -1)
        printf("No page visited\n");
    else
        printf("Current page: %s\n", history[top]);
}

void display()
{
    int i;

    for (i = top; i >= 0; i--)
        printf("%s\n", history[i]);
}

int main()
```

```
{
    int choice;
    char page[50];

    while (1)
    {
        printf("\n1.Visit  2.Back  3.Current  4.History  5.Exit\n");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter webpage: ");
                scanf("%s", page);
                visit(page);
                break;

            case 2:
                back();
                break;

            case 3:
                current();
                break;

            case 4:
                display();
                break;

            case 5:
                return 0;
        }
    }
}
```

OUTPUT:

```
1.Visit  2.Back  3.Current  4.History  5.Exit
1
Enter webpage: 34
Visited: 34

1.Visit  2.Back  3.Current  4.History  5.Exit
3
Current page: 34

1.Visit  2.Back  3.Current  4.History  5.Exit
2
No previous page

1.Visit  2.Back  3.Current  4.History  5.Exit
```

```
4
34

1.Visit  2.Back  3.Current  4.History  5.Exit
1
Enter webpage: 4
Visited: 4

1.Visit  2.Back  3.Current  4.History  5.Exit
2
Back to: 34

1.Visit  2.Back  3.Current  4.History  5.Exit
5
--------------------------------------------------------------------------------
--------
4.Sort a stack using temporary stack
#include <stdio.h>

#define MAX 10

int stack[MAX], temp[MAX];
int top = -1, ttop = -1;

void push(int x)
{
    stack[++top] = x;
}

int pop()
{
    return stack[top--];
}

void sortStack()
{
    int x;

    while (top != -1)
    {
        x = pop();

        while (ttop != -1 && temp[ttop] > x)
        {
            stack[++top] = temp[ttop--];
        }

        temp[++ttop] = x;
    }
```

```
    while (ttop != -1)
    {
        stack[++top] = temp[ttop--];
    }
}

void display()
{
    int i;

    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);

    printf("\n");
}

int main()
{
    int n, i, x;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &x);
        push(x);
    }

    sortStack();

    printf("Sorted stack:\n");
    display();

    return 0;
}
```

```
OUTPUT:
Enter number of elements: 5
Enter elements:
12
34
56
7
90
Sorted stack:
7 12 34 56 90
--------------------------------------------------------------------------------
------------------------------
```

5.Stock stack plan

```
#include <stdio.h>

#define MAX 100

int main()
{
    int price[MAX], span[MAX], stack[MAX];
    int n, top = -1;
    int i;

    printf("Enter number of days: ");
    scanf("%d", &n);

    printf("Enter stock prices:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &price[i]);

    stack[++top] = 0;
    span[0] = 1;

    for (i = 1; i < n; i++)
    {
        while (top != -1 && price[stack[top]] <= price[i])
            top--;

        if (top == -1)
            span[i] = i + 1;
        else
            span[i] = i - stack[top];

        stack[++top] = i;
    }

    printf("Stock Span:\n");

    for (i = 0; i < n; i++)
        printf("%d ", span[i]);

    return 0;
}
```

OUTPUT:

```
Enter number of days: 5
Enter stock prices:
12
34
6
6
78
Stock Span:
```

```
1 2 1 2 5
--------------------------------------------------------------------------------
------------------------------6.Nearest Element to Right with Greater Frequency
#include <stdio.h>

#define MAX 100

int main()
{
    int a[MAX], freq[MAX], stack[MAX], result[MAX];
    int n, top = -1;
    int i, j;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);


    for (i = 0; i < n; i++)
    {
        freq[i] = 0;

        for (j = 0; j < n; j++)
        {
            if (a[i] == a[j])
                freq[i]++;
        }
    }


    for (i = n - 1; i >= 0; i--)
    {
        while (top != -1 && freq[stack[top]] <= freq[i])
            top--;

        if (top == -1)
            result[i] = -1;
        else
            result[i] = a[stack[top]];

        stack[++top] = i;
    }

    printf("Output:\n");

    for (i = 0; i < n; i++)
        printf("%d ", result[i]);
```

```
    return 0;
}
```
OUTPUT:
```
Enter number of elements: 6
Enter elements:
3
4
2
1
3
4
```
Output:
```
-1 -1 3 3 -1 -1
```


7.Pattern using I and D
```
#include <stdio.h>
#include <string.h>

#define MAX 20

int stack[MAX];
int top = -1;

void push(int x)
{
    stack[++top] = x;
}

int pop()
{
    return stack[top--];
}

int main()
{
    char pattern[MAX];
    int i, n, num = 1;

    printf("Enter pattern containing I and D: ");
    scanf("%s", pattern);

    n = strlen(pattern);

    for (i = 0; i <= n; i++)
    {
        push(num++);

        if (i == n || pattern[i] == 'I')
```

```
        {
            while (top != -1)
                printf("%d", pop());
        }
    }

    return 0;
}
```

OUTPUT:

```
Enter pattern containing I and D: IDIDD
132654
```

8.Minimum sum of two numbers

```
#include <stdio.h>

int main()
{
    int a[100], n;
    int i, j, temp;
    int num1 = 0, num2 = 0;

    printf("Enter number of digits: ");
    scanf("%d", &n);

    printf("Enter digits:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);


    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (a[i] > a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }


    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0)
            num1 = num1 * 10 + a[i];
        else
            num2 = num2 * 10 + a[i];
```

```
    }

    printf("Minimum sum = %d\n", num1 + num2);

    return 0;
}
```

OUTPUT:

```
Enter number of digits: 5
Enter digits:
2
4
5
7
1
Minimum sum = 172
```

9.First negative integer in every window

```
#include <stdio.h>

int main()
{
    int a[100], queue[100];
    int n, k;
    int front = 0, rear = -1;
    int i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter window size K: ");
    scanf("%d", &k);



    for (i = 0; i < n; i++)
    {

        if (a[i] < 0)
            queue[++rear] = i;


        if (front <= rear && queue[front] <= i - k)
            front++;
```

```
        if (i >= k - 1)
        {
            if (front <= rear)
                printf("%d ", a[queue[front]]);
            else
                printf("0 ");
        }
    }

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 9
Enter elements:
-1
 2
 3
-4
 5
 6
-7
 8
 9
Enter window size K: 3
-1 -4 -4 -4 -7 -7 -7
```

10.Rotten Oranges-Queue

```
#include <stdio.h>

#define MAX 100

struct Node
{
    int row;
    int col;
    int time;
};

int main()
{
    int a[MAX][MAX];
    struct Node queue[MAX * MAX];

    int m, n;
    int front = 0, rear = -1;
    int i, j;
    int time = 0;
    int fresh = 0;
```

```
printf("Enter rows and columns: ");
scanf("%d %d", &m, &n);

printf("Enter matrix:\n");

for (i = 0; i < m; i++)
{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &a[i][j]);

        if (a[i][j] == 1)
            fresh++;

        if (a[i][j] == 2)
        {
            rear++;
            queue[rear].row = i;
            queue[rear].col = j;
            queue[rear].time = 0;
        }
    }
}


int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};

while (front <= rear)
{
    struct Node current;
    current = queue[front++];

    if (current.time > time)
        time = current.time;

    for (i = 0; i < 4; i++)
    {
        int nr = current.row + dr[i];
        int nc = current.col + dc[i];

        if (nr >= 0 && nr < m &&
            nc >= 0 && nc < n &&
            a[nr][nc] == 1)
        {
            a[nr][nc] = 2;
            fresh--;

            rear++;
```

```
                queue[rear].row = nr;
                queue[rear].col = nc;
                queue[rear].time = current.time + 1;
            }
        }
    }

    if (fresh > 0)
        printf("-1\n");
    else
        printf("Minimum time = %d\n", time);

    return 0;
}
```

```
OUTPUT:
Enter rows and columns: 3

3
Enter matrix:
2
1
0
1
1
2
0
1
1
Minimum time = 2
```